Stegstr

Steganographic social networking — fork of brunkstr/Stegstr. Contest entry — Muhammad Akif Janjua — github.com/akifjanjua/Stegstr

Summary

This fork fixes **9 bugs** in total. **5 are confirmed pre-existing in upstream** — each verified by building and running a pristine clone of `brunkstr/Stegstr` @ `ad2e10e` directly (not inferred from reading the code): bugs #1, #2, #4, #5, #6. **3 more were found and fixed in this fork's own new work** (bugs #3, #7, #8 — code that doesn't exist in upstream at all: the JPEG/QIM encoder and the Nostr publish-confirmation feature). **1 (bug #9) is neither**: its faulty line is identical to upstream's, but the bug is only reachable because of this fork's own dual-encoder decode path.

By severity (as recorded per-bug in BUGS.md): **4 Critical** (#1, #5, #6, #8), **2 High** (#2, #4), **3 Medium** (#3, #7, #9).

Separately, **5 gaps are documented but intentionally left unfixed**, each disclosed in BUGS.md rather than omitted:

- NIP-01 `CLOSED`/`NOTICE` relay messages are not handled at all.
- No offline outbox exists for non-zap events — an unsent post/reply/DM/etc. is lost on restart.
- Bug #2's residual clamping case (a block where no LH value avoids clamping) is mathematically possible but was not observed across 60+ round-trips of this fork's own encoder+decoder.
- Phase 4's adaptive QIM delta still fails a literal solid-color cover on the harshest simulated platform — 1 of 125 tested combinations.
- cargo-fuzz scaffolding for `decode_any()` is committed but was never actually run, due to a Windows/MSVC libFuzzer linker limitation (`LNK1561`).

Plus repo/toolchain fixes: 8 npm vulnerabilities fixed (1 critical), a broken `mobile-android` git submodule removed, MSRV 1.88.0 declared and verified, outdated GitHub Actions bumped. **Separately, full AI agent operability** was added and shipped in **v0.1.2**: `--json` + schemas on 5 commands, documented exit codes, a new `calibrate` command, and an MCP server — see page 2.

The three headline bugs

Full repro steps, root cause, and regression tests for all 9 bugs are in BUGS.md. These three are the most severe and most representative — one upstream security hole, one upstream data-corruption bug, one upstream image-destruction bug.

Bug	What it was	Commit
#6 — Nostr signature spoofing	Received Nostr events were never cryptographically verified. Any relay, or a MITM, could inject events that display in the UI as genuinely posted by any pubkey — including someone else's. Confirmed pre-existing upstream (identical `onmessage` handler, byte-for-byte). Fixed with `verifyEvent()`; verified not overly strict against 100 real events pulled from two live public relays (100/100 accepted, 0 false rejections).	`fea5984`
#1 — Silent payload corruption, default encoder	The default `embed` (no flags) returned wrong bytes on decode for any cover ≥256px with a payload over ~9 bytes — no error, just wrong data. Root cause: `decode()` tried a whole-image transform before the correct tile-aligned one. Confirmed pre-existing upstream by building and running a pristine clone directly.	`699f4cb`
#5 — Image-destruction bug	`encode()` read every non-leftmost tile's pixels from the wrong source column, visibly wrecking the cover image for any width >256px (payload still decoded fine, which is why no prior test caught it). Measured PSNR 8.96 dB / SSIM 0.75 on a 512×512 gradient before the fix, versus 30–67 dB everywhere else. Confirmed pre-existing upstream.	`08bd234`

Live platform results — stated precisely

“Survives recompression” and “passes through untouched” are different claims. Both are documented separately, not conflated.

Platform	Result	What was actually shown
Instagram	**Live, genuine survival**	Sent through real Instagram: file size changed (**+9.1%, 48,241 → 52,609 bytes**) — Instagram actually re-encoded it — and the payload still decoded correctly.
WhatsApp	**Live, but pass-through, not survival**	Sent through real WhatsApp: came back byte-identical. A control test (an untouched, unembedded original sent the same way) came back **~76% smaller**, proving WhatsApp's pipeline *does* recompress — just not this file. The stego output is small/compliant enough that WhatsApp's own pipeline treats it as a no-op, so this is not evidence the payload survived a recompression pass.
Telegram	**Simulated only**	Same code path validated for WhatsApp/Instagram, run through a local simulator matching Telegram's known resize/quality settings — not sent through a real Telegram client.

Full methodology and the simulated 45/45 channel matrix (9 cover types × 5 platform profiles): `channel_simulator/BASELINE_RESULTS.md` (referenced from README.md).

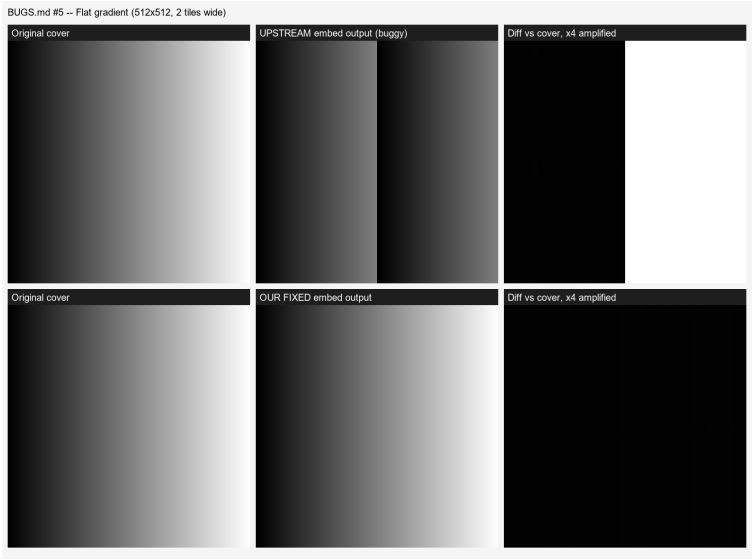
AI agent operability (new in v0.1.2)

The CLI and an MCP server are first-class here, not an afterthought — every claim below is verified against the real compiled binary in `src-tauri/tests/cli_json_schema.rs` and `tests/e2e/agent_smoke.sh`, not just documented.

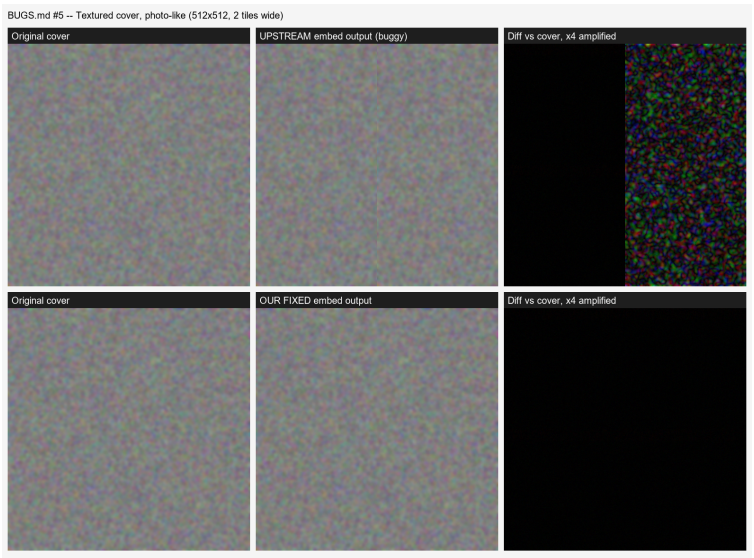
- `--json` **on decode/detect/embed/post/calibrate** — exactly one JSON object on stdout, no prose mixed in. Schemas committed at `schema/cli/*.schema.json`, validated against the real binary in tests.
- **Documented, source-verified exit codes** — 0 success, 1 generic, 2 capacity exceeded, 3 no payload found, 4 decryption failure, 5 malformed input. Each classification is tied to an exact, cited error string from the encoder/decoder/crypto code, not keyword-sniffing; all 4 non-generic codes triggered for real in tests, not just asserted.
- `calibrate` **(new command)** — a pure-Rust JPEG marker parser infers a platform's re-encode pipeline (resize rule, JPEG quality, chroma subsampling, progressive/baseline, metadata stripped) from a sent/received pair. Verified against this repo's own real captured evidence: exact JPEG quality 80 (Instagram) and 92 (WhatsApp control) recovered byte-exact, and it independently inferred WhatsApp's real ~1600px resize cap from pixel dimensions alone.
- `mcp` **(new command)** — a stdio MCP server on the official `rmcp` SDK, exposing embed/decode/detect/calibrate as tools with typed schemas. Verified with real `initialize`/`tools/list`/`tools/call` round trips, not just started and assumed working.
- `skill/stegstr/` **rewritten**, every command block actually run against the compiled binary — caught a real stale-claim risk before it shipped (a wrapper-vs-bundle confusion in the example workflow). `tests/e2e/agent_smoke.sh` added: full headless flow, 10/10 passing.

Evidence: bug #5, before/after (upstream vs. fixed)

Each image below is a 512×512 cover embedded with the same 188-byte payload, encoded once with pristine upstream's buggy encoder and once with this fork's fixed encoder. PSNR/SSIM values are exact figures from BUGS.md's before/after measurement table.



Smooth horizontal gradient (most dramatic case) — before: PSNR 8.96 dB / SSIM 0.7521 | after: PSNR 51.16 dB / SSIM 0.9956. Top row: cover → pristine upstream's buggy encoder output → diff vs. cover (4× amplified). Bottom row: cover → this fork's fixed encoder output → diff (4× amplified). (`docs/evidence/bug5_flat_gradient.png`)



Textured/noisy cover approximating a real photo — before: PSNR 31.34 dB / SSIM 0.8430 | after: PSNR 50.41 dB / SSIM 0.9944. Top row: cover → pristine upstream's buggy encoder output → diff vs. cover (4× amplified). Bottom row: cover → this fork's fixed encoder output → diff (4× amplified). (`docs/evidence/bug5_photo.png`)

A third cover (pale blue sky gradient, 44.89→49.19 dB) is in the same evidence set but omitted here for space; see `docs/evidence/` in the repo for all three.

What was not verified

Disclosed directly rather than left implicit:

- Telegram: code path shares its logic with the confirmed WhatsApp/Instagram sends, but was only run through a local simulator matching Telegram's known settings — never a real Telegram client.
- Linux AppImage: checksum-verified and correct size, but launching it was not tested — built and verified from a Windows machine with no Linux environment available.
- cargo-fuzz on `decode_any()`: scaffolding (`fuzz/Cargo.toml`, `fuzz_targets/decode_any.rs`) is committed and ready to run, but was never actually executed — blocked by a Windows/MSVC libFuzzer linker limitation (`LNK1561: entry point must be defined`), a known toolchain issue unrelated to this codebase. Deferred to Linux, cargo-fuzz's fully-supported platform.

- NIP-01 `CLOSED`/`NOTICE` relay messages: confirmed via a mock relay that the client receives them but takes no action — no re-subscribe, no user-visible signal.
- Offline outbox: only zap events persist to `localStorage` and auto-flush on reconnect. Every other event type (posts, replies, DMs, likes, follows, reactions, profile/contact edits) has no retry or persistence if unsent.
- Binary-safe stdin/stdout piping for `embed`/`detect`, and a `--seed` flag for deterministic output where randomness is used: both were part of the original AI-agent-operability brief but weren't built in v0.1.2 — flagged as open scope, not silently dropped.

Downloads & repository

v0.1.2 (current) — no Rust, no Node, no build step. All builds run in GitHub Actions from a clean clone.

Platform	Download
Windows	Stegstr-Windows.exe or .msi
macOS	Stegstr-macOS.dmg
Linux	Stegstr-Linux.AppImage or .deb

Verified before publishing, not just built: the Windows download was installed and launched for real, and the bundled CLI was round-tripped (embed → decode, byte-exact) before this link went out. `SHA256SUMS.txt` is in the release for independent verification.

Repository: https://github.com/akifjanjua/Stegstr

Latest release: https://github.com/akifjanjua/Stegstr/releases/latest

Bugs found and fixed (BUGS.md): https://github.com/akifjanjua/Stegstr/blob/main/BUGS.md

Robustness report: https://github.com/akifjanjua/Stegstr/blob/main/ROBUSTNESS_REPORT.md

Evidence images: https://github.com/akifjanjua/Stegstr/tree/main/docs/evidence

Agent skill (MCP/CLI for agents): https://github.com/akifjanjua/Stegstr/tree/main/skill/stegstr

CLI JSON schemas: https://github.com/akifjanjua/Stegstr/tree/main/schema/cli